

AI for Economic Research

Lec. 9: Agentic AI, Research Workflows, and Claude Code

Zhigang Feng

Spring 2026

Lecture Agenda

1. Why economists should care
2. What web chat can and cannot do
3. Getting started: installation + Git
4. Claude Code: under the hood
5. Your first session: the workflow
6. The 5-pillar workflow
7. The homotopy workflow
8. Which tool fits which task
9. How a project is organized
10. Mistakes, limits, and security
11. The context dilemma
12. Lecture 10 preview + wrap-up

Teaching claim: agents reduce execution cost and increase iteration speed, but economists still own specification, validation, and interpretation.

Framing

Why agents matter for economists

Why Economists Should Care

The bottleneck in applied research is often not ideas. It is execution.

- Turning a good question into clean code, diagnostics, tables, and figures is slow
- Many research tasks are repetitive: reshape data, rerun specifications, refactor scripts, update plots
- Agentic tools compress the time from *idea* to *first defensible result*
- That raises the return to the skills economists uniquely provide: model design, institutional knowledge, and judgment

The promise is not “AI does research for you.” The promise is faster implementation and faster iteration around a question you still have to define and defend.

What Modern Web Chat Already Does Well

As of April 2026, a strong web-chat product is already useful for economists.

- Brainstorming model setups, empirical strategies, and robustness checks
- Summarizing papers, referee reports, and lecture notes
- Inspecting uploaded files and explaining code snippets
- Sometimes browsing the web or running limited sandboxed tools
- Helping you draft a specification before you open your editor

So the weak baseline is not “a browser cannot do anything.” The weak baseline is a *single chat session without a durable project harness*.

What Still Breaks Without a Project Harness

A single web-chat session often fails at the exact points where research projects become real.

1. **Durable context is weak.** Yesterday's conventions and corrections are not reliably embedded in today's work
2. **Reusable workflow is weak.** Your validation checklist lives in prompts, not in inspectable files
3. **Controlled execution is weak.** The model is not operating inside your actual project tree with your actual scripts and outputs
4. **Audit trail is weak.** It is hard to inspect which files, commands, and checks produced the result

That is the real contrast. Agents are not magic because they are “smarter.” They are useful because they can operate inside a structured project with files, tools, and repeatable workflows.

What Makes Something Agentic?

Four capabilities matter for research work:

- **Tool use:** run Python, shell commands, plotting libraries, and tests
- **Multi-step execution:** carry out a sequence such as read → clean → estimate → plot
- **Iteration on real output:** see tracebacks, failed checks, and wrong figures, then adjust
- **Persistence through project files:** reuse context, conventions, and saved workflows across sessions

Economic translation: an agent is closer to a diligent research assistant with tool access than to a one-shot text generator.

Spectrum: Chat, IDE Assistant, Terminal Agent

	Web chat	IDE assistant	Terminal agent
Best at	thinking, drafting	editing local code	end-to-end execution
Context	conversation + uploads	open files	full project tree
Typical role	architect's whiteboard	coding copilot	workflow operator
Economic example	design the specification	clean one regression file	run the full pipeline

This lecture uses Claude Code as the concrete terminal-agent example, but the workflow generalizes to other terminal agents.

The Bridge: How a Chatbox Becomes a Terminal Agent

One key technical change: modern LLMs (post-2023) support **structured tool calls**. Instead of writing “you should run `python solve.py`,” the model emits a machine-readable request:

```
{ "tool": "Bash", "command": "python aiyagari_v0.py" }
```

A **harness**—software installed on your Mac—executes this and returns the real output. The model reads it and decides what to do next. This loop is what makes something “agentic.”

	claude.ai (browser)	Claude Code (terminal)
LLM model	same (Opus 4.6)	
Harness	none (sandboxed)	local CLI on your Mac
Can run Python?	no	yes— <i>your</i> Python, <i>your</i> machine
Sees real errors?	no	yes—loops back automatically

Same brain, different arms. The terminal agent is not a smarter model—it is the *same* model given real tools. The harness is what flips the switch from text-only to text + actions.

Getting Started

Installing Claude Code and setting up version control

Installing Claude Code on Your Mac

Five steps (about five minutes total):

1. **Install Node.js** (required runtime)

- Easiest: `brew install node` (requires Homebrew: `brew.sh`)
- Alternative: download from nodejs.org (v18 or later)

2. **Install Claude Code:**

```
npm install -g @anthropic-ai/claude-code
```

3. **Verify:** `claude -version`

4. **Subscribe** at claude.ai

- **Pro** (\$20/mo) — Claude Sonnet 4.5, adequate for most tasks
- **Max** (\$100/mo) — Claude Opus 4.6, stronger reasoning, higher limits

5. **First launch:** `cd your-project-folder` then `claude`

A browser window opens for OAuth. Log in once—done.

Mac + OneDrive note: always `cd` into your project folder before running `claude`. Running from a `/Volumes/...` path can trigger permission errors.

Why Git, Not OneDrive or Dropbox

The risk without Git:

- Claude Code can edit dozens of files autonomously in one session
- OneDrive and Dropbox sync by *copying*—two simultaneous writes create corrupt merge files
- No undo: if Claude makes bad changes, there is no “revert” without Git
- No audit trail: you cannot see what changed or when

What Git gives you:

- `git diff` shows exactly what Claude changed before you accept it
- `git checkout -- .` reverts all uncommitted changes instantly—your emergency brake
- `git log` shows every change with a timestamp and message
- Works offline; cloud sync races with Claude’s rapid file writes

Practical rule: store large data files on OneDrive. Store code and scripts in Git. Never use the same tool for both.

First Git Setup: Three Minutes

One-time global setup:

```
git config --global user.name "Your Name"
git config --global user.email "you@email.com"
"
```

Per-project setup:

```
cd my-project
git init
echo "data/ output/ *.pdf" > .gitignore
git add .gitignore
git commit -m "initial setup"
```

You do not need to know Git deeply. Three commands—`git add -A`, `git commit -m "..."`, `git checkout -- .`—are enough to work safely with Claude Code.

The one rule:

- Commit *before* every Claude session
- Commit *after* validating results
- That is enough to start safely

Emergency brake:

```
git checkout -- .
```

Undoes *all* uncommitted file changes instantly.
Your safety net when Claude goes wrong.

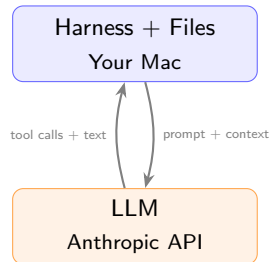
Claude Code: Under the Hood

File system, startup, permissions, and key commands

Three Components of Claude Code

Claude Code has three physically distinct parts:

1. **The Harness** (lives on your Mac)
CLI software you install. Reads your files, runs shell commands, manages the conversation loop.
2. **The LLM** (lives on Anthropic's servers)
Every message is bundled with conversation history and sent via HTTPS to the API. The “thinking” happens in the cloud.
3. **Your Files** (live on your Mac)
Python scripts, data CSVs, LaTeX—all on your disk.
Code executes on your hardware with your packages.



Key: code runs locally.
Only the *conversation* traverses the network.
Your data never uploads.

The Mac File Hierarchy: Global vs. Local

Two layers of configuration. **Local overrides global.**

Path	Scope	What lives here
<code>~/.claude/settings.json</code>	Global	Default permissions, model preference
<code>~/.claude/MEMORY.md</code>	Global	Cross-project memory (persists forever)
<code>~/.claude/skills/</code>	Global	Skills available in every project
<code><project>/CLAUDE.md</code>	Local	Project brain, read every session
<code><project>/.claude/settings.json</code>	Local	Project permissions (overrides global)
<code><project>/.claude/skills/</code>	Local	This project's slash commands
<code><project>/.claude/agents/</code>	Local	Specialized reviewer roles
<code><project>/.claude/rules/</code>	Local	Always-on guardrails

On a Mac: `~` = `/Users/yourname`. To inspect: `ls ~/.claude/`

Rule: global `~/.claude/` is your personal defaults (apply everywhere). Local `.claude/` is the project's rules (apply only here).

Initialization: What Happens When You Type `claude`

Five steps run **before you type your first message**:

1. **Read global config** — loads `~/.claude/settings.json`: permission defaults, model preference
2. **Detect project context** — searches current directory (and parents) for `CLAUDE.md` and `.claude/`
3. **Load project brain** — reads `CLAUDE.md` into the context window
4. **Auto-load all rules** — every `.claude/rules/*.md` file is loaded silently
5. **Ready** — context window pre-populated before you say anything

Analogy: step 4 is a professor re-reading lecture notes before class. Students arrive; context is already loaded.

Cost warning: steps 3–4 consume tokens before you type anything. Keep `CLAUDE.md` and rules files concise.

The Permission System and Local Code Execution

Claude's toolbox:

Tool	What it does
Bash	Run any shell command
Read	Read a file from disk
Write	Create or overwrite a file
Grep/Glob	Search content / find files
Agent	Spawn a child Claude instance

The permission model:

- **Default:** Claude asks before every Bash call. You see the exact command first.
- **Whitelist:** add trusted commands to `.claude/settings.json`:

```
"allowedTools": [  
  "Bash(python *)",  
  "Bash(git *)" ]
```

- **Bypass-permissions:** skips all prompts. Use only when you fully trust the task.

“Which Python ran this?” (common confusion):

- Claude's Bash inherits your shell's \$PATH
- If you ran `conda activate myenv` before claude, *that* Python runs
- Add a `which python` check to your `CLAUDE.md` to confirm

Key Modes: Plan, Effort, and Session Control

Command / Key	What it does	When to use
Shift+Tab	Plan mode: Claude writes a plan, does NOT execute. Waits for approval.	Start every non-trivial task here
/effort max	Maximum reasoning depth: highest quality, slower, costs more	Novel problems, economic correctness
/effort default	Balanced quality and speed	Routine edits, simple queries
/cost	Show tokens used + estimated USD for this session	Budget check before long tasks
Esc	Interrupt Claude mid-task immediately	When it's going the wrong way
/compact	Summarize conversation to free context window	After ~20 turns; between phases
/clear	Wipe context, start fresh	New unrelated task

The effort–cost tradeoff:

- /effort max + long context $\approx$ \$5–15 per complex session (Opus 4.6 pricing)
- Spend effort on *decisions*; save it on *execution*

Your First Session

The recommended workflow from scratch

Start Here: The Recommended Workflow for New Users

0. Before Claude—pen + paper

Write one sentence: “I want to [compute X] using [method Y] for [question Z].” If you cannot write this, Claude cannot help yet.

1. Enter plan mode (Shift+Tab)

Describe your goal. Claude drafts a plan. *Nothing runs yet.*

2. Review and approve

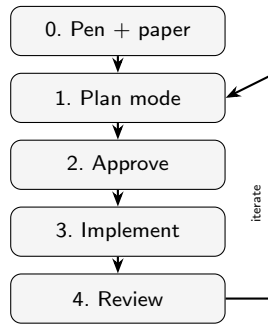
Read the plan. Ask questions. Revise. Type “approved.” *This 2-minute step saves 20 minutes of debugging.*

3. Implement

Claude executes the approved plan. Press Esc to stop if wrong.

4. Review output

Does it make economic sense? Re-derive one

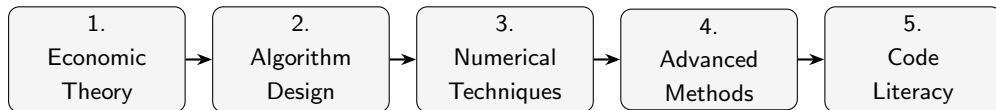


Most common beginner mistake: skipping Step 0. Without a clear goal, Claude will confidently build the wrong thing.

The 5-Pillar Workflow

What stays human when agents get stronger

The 5-Pillar Workflow



The throughline of the lecture:

- Pillars 1–4 determine what should be built
- Pillar 5 lets you verify what was actually built
- If the pillars are weak, agent output becomes faster nonsense

Pillars 1 and 2: Theory and Algorithm

Pillar 1: Economic theory

- State the objective, constraints, information structure, and equilibrium concept
- Decide what constitutes the economic object of interest: policy, distribution, moment, elasticity
- Write down limiting cases before any code exists

Pillar 2: Algorithm design

- Choose between Bellman iteration, time iteration, EGM, perturbation, simulation, or estimation routines
- Decide what the acceptance criterion is: convergence, speed, or benchmark recovery
- Convert theory into a clear workflow the agent can implement

Aiyagari anchor: the agent can write VFI code, but only you know whether VFI, Howard improvement, or EGM is the right choice.

Pillars 3 and 4: Numerics and Advanced Methods

Pillar 3: Numerical techniques

- Grid design, interpolation, discretization of shocks, simulation, and stability checks
- Recognition of failure modes: kinks, boundary effects, coarse grids, slow convergence

Pillar 4: Advanced methods

- Projection, perturbation, global solution methods, and high-dimensional approximations
- Knowing when the agent's default method is convenient but economically wrong

Economics point: a model can run cleanly and still be numerically misleading. Agents reduce coding cost; they do not remove approximation error.

Pillar 5: Code Literacy and Division of Labor

You provide	Agent provides
Question, model, and identification logic	Fast code generation and refactoring
Algorithm choice and benchmark design	Boilerplate implementation and debugging
Knowledge of the data and institutional setting	File edits, script writing, and test runs
Economic interpretation and seminar defense	Iteration on visible errors and outputs

Research version of the slogan: the agent can be a strong RA, but you still have to be the principal investigator.

Validation Is Your Referee Checklist

Every iteration needs both technical and economic checks.

Technical checks

- Convergence achieved
- No NaN or Inf values
- Policies stay off unintended grid boundaries
- Diagnostics and residuals within tolerance
- Script reproduces the same output twice

Economic checks

- Limiting cases recover known results
- Comparative statics have the right sign
- Magnitudes are plausible
- Inference uses the right standard errors and sample
- You can explain the result in plain economic language

If you cannot defend the output in a seminar, you are not done.

The Homotopy Workflow

Build from simple to complex, validate, then extend

The Homotopy Workflow

In numerical analysis, homotopy means solving an easier problem first and then deforming it toward the target problem.

- Start from a version with clear intuition and known checks
- Use that validated baseline as the starting point for the next version
- Attribute new bugs to the new margin you just added
- Avoid asking the agent to solve the full complicated problem in one jump

Aiyagari example:

V0 deterministic $\rightarrow$ V1 income risk $\rightarrow$ V2 stationary distribution $\rightarrow$ V3 equilibrium

Why this matters for agentic coding: the agent performs best when each step is bounded, testable, and attached to an existing working baseline.

Homotopy in Practice: Build from Simple to Complex



This replaces the fantasy prompt “solve my model.”

- Start with a problem you can state and benchmark
- Give the agent bounded tasks with visible acceptance criteria
- Add one new ingredient at a time

Step 1: Design the Economic Problem

Before any code, write down the model in economist language.

- Research question in one sentence
- Objective and constraints
- Equilibrium definition if applicable
- Parameters and economic motivation
- Outputs you will treat as evidence

Aiyagari anchor:

$$\max \mathbb{E}_0 \sum_{t=0}^{\infty} \beta^t u(c_t) \quad \text{s.t.} \quad c_t + a_{t+1} = (1+r)a_t + wz_t, \quad a_{t+1} \geq \underline{a}$$

Reason: a vague model description produces code that may be internally clean but economically mis-specified.

Step 2: Write the Specification File

The specification file is the contract between you and the agent.

A compact Aiyagari spec might include:

1. CRRA utility with chosen β and σ
2. Income process and discretization method
3. Asset grid shape near the borrowing constraint
4. Solution method and convergence tolerance
5. Required outputs: policies, stationary distribution, inequality statistics

Best practice: ask the agent to restate the specification in its own words before it writes code. That catches misunderstandings when they are still cheap.

Step 3: Choose the Algorithm Explicitly

Pseudocode is where you catch design mistakes before implementation.

```
Initialize grid and a guess for V_old
Repeat until convergence:
  For each asset point a_i:
    For each feasible next asset a':
      compute  $u(c) + \beta * EV_{old}(a')$ 
      choose the maximizing a'
    update policies and diagnostics
Return value function, policy functions, and checks
```

The point is not elegance. The point is that you and the agent agree on the algorithm before Python or Julia details begin to hide conceptual errors.

Step 4: Build V0 First

V0 is the simplest version with a benchmark you understand.

Aiyagari V0: deterministic income

- If $\beta(1+r) = 1$, savings policy should sit on the 45-degree line
- If $\beta(1+r) < 1$, the agent decumulates toward the borrowing constraint
- If $\beta(1+r) > 1$, assets grow

Discipline: do not ask the agent to add income risk, stationary distributions, and equilibrium search before V0 already passes the simple checks.

Steps 5 and 6: Validate, Then Extend One Margin at a Time

Version	New ingredient	Required benchmark
V0	Deterministic household	analytical cases
V1	Markov income risk	setting shock variance to zero recovers V0
V2	Stationary distribution	mass sums to 1 and behaves sensibly at the constraint
V3	General equilibrium	market clearing and stable ag- gregate statistics

Why this works with agents: when a new version breaks, both you and the agent know where to look.

Why This Workflow Works for Economists

- It keeps theory, computation, and validation aligned
- It gives the agent bounded tasks instead of open-ended wishes
- It preserves an audit trail of how the baseline was constructed
- It mirrors how good research assistants are actually trained: start with a simple task, verify it, then widen scope

Main lesson: the workflow is the real product. The particular agent you use can change.

Tool Choice

Which tool fits which research task

Choose the Tool That Matches the Task

Tool type	Use it for	Economic example
Web chat	framing and explanation	clarify identification logic, draft a model summary
IDE assistant	local code editing	vectorize one function or explain one traceback
Terminal agent	multi-step execution	restructure a project, run scripts, generate diagnostics, and iterate

As of April 2026: Claude Code, Codex CLI, Gemini CLI, and Aider are examples of terminal-agent workflows.

Why the Terminal Agent Often Becomes the Workhorse

- It can operate on the real project rather than on copied fragments
- It can run the code, inspect errors, and update multiple files
- It works well when the task is bounded but longer than one edit
- It still leaves the economist in the approval and validation loop

For empirical research, this is usually the sweet spot: enough autonomy to save time, not so much autonomy that you stop checking the economics.

Project Organization

How a terminal-agent project stays organized

Why Files on Disk Matter

The project harness turns today's chat into tomorrow's reusable workflow.

- Conventions live in files, not in your memory
- Validation steps become reproducible lab procedure
- New sessions can start from the project's actual state
- Coauthors and students can inspect what the workflow is supposed to be

Economist translation: the harness is like turning tacit RA practice into written lab protocol.

Minimal Project Tree

```
1 my-aiyagari-project/  
2 +-- CLAUDE.md  
3 +-- .claude/  
4 |   +-- skills/  
5 | |   +-- solve-model/  
6 | |       +-- SKILL.md  
7 | |       +-- references/  
8 | |           +-- aiyagari-guide.md  
9 | +-- agents/  
10 | |   +-- code-reviewer.md  
11 | |   +-- domain-reviewer.md  
12 | |   +-- verifier.md  
13 | +-- rules/  
14 | |   +-- iterative-workflow.md  
15 | +-- settings.json  
16 +-- scripts/  
17 +-- output/  
18 +-- quality_reports/
```

This is enough to explain the workflow without burying students in infrastructure.

CLAUDE.md: The Team-Facing Project Memory

CLAUDE.md is the main shared instruction file for the project.

```
# CLAUDE.md
Project: Aiyagari baseline for lecture 9
Language: Python (NumPy/SciPy)
Goal: Build and validate v0, v1, and v2 in sequence
Conventions: no hardcoded paths; save outputs to output/
Checks: do not claim success before benchmarks pass
```

Why students should care: it captures the kind of project context you would otherwise re-explain every session.

CLAUDE.md, Auto Memory, and Optional Course Logs

Three different persistence ideas should not be confused.

Mechanism	Who writes it	Why it exists
CLAUDE.md	you / team	durable project instructions and conventions
Auto memory	the tool	per-project learned patterns and corrections
Optional course log	you / course	explicit progress notes such as MEMORY.md or progress.md

Course recommendation: rely on CLAUDE.md for shared rules, treat auto memory as helpful but not authoritative, and keep explicit logs when you want students to inspect the evolution of a project.

Skills, Agents, and Rules: Three Different Jobs

Object	Economist analogy	What it does
Skill	saved RA protocol	reusable workflow such as solve, validate, review
Agent	specialist reviewer	a focused role: code quality, economics, verification
Rule	lab convention	always-on instruction such as plan first or save outputs here

This distinction matters because the student should know where to encode each idea.

Plain English vs. Encoded Skill

Plain English

Implement the Aiyagari model with income risk and show the benchmark checks.

Use when

- Exploring
- Doing a one-off task
- Still deciding the workflow

Encoded skill

```
/solve-model aiyagari v1
```

Use when

- The workflow should repeat exactly
- A student team needs one standard procedure
- Validation must be consistent across versions

Exploration starts in English. Reproducibility lives in skills.

Skill File Structure

```
.claude/skills/solve-model/  
  SKILL.md  
  references/  
    aiyagari-guide.md
```

SKILL.md stores the repeatable protocol:

- what the command is called
- what steps it should follow
- what checks must pass before it reports success

Reference files store model-specific detail that would clutter the main instruction file.

Agents as Specialist Reviewers

Agent files are best understood as specialist review roles.

- `code-reviewer.md`: vectorization, file hygiene, implementation quality
- `domain-reviewer.md`: Euler equations, benchmark logic, economic consistency
- `verifier.md`: run the script, inspect diagnostics, check output exists

Economist analogy: one RA checks code style, another checks economics, and another reruns the table.

Subagents as Parallel Reviewers

Subagents matter because review tasks can happen in parallel.

- The main agent writes or edits code
- A code-reviewer and domain-reviewer can inspect the result at the same time
- The main conversation receives both reports and decides what to fix

Why students should care: the gain is not philosophical. It is wall-clock time. Focused review roles are faster and often clearer than one overloaded prompt.

Rules, Hooks, and Settings

Object	Loaded when	Typical use
Rules	during project work	always follow a planning or verification convention
Hooks	on tool events	run a formatter or log a command mechanically
Settings	at session start	permissions and project-level behavior

Practical distinction: if it is natural-language guidance, it usually belongs in a rule or skill. If it must happen mechanically every time, a hook is safer.

Tracing Aiyagari End-to-End

How the workflow maps onto files and review loops

Aiyagari Recap

Household problem

$$c_t + a_{t+1} = (1 + r)a_t + wz_t, \quad a_{t+1} \geq \underline{a}$$

with idiosyncratic productivity z_t following a Markov chain.

Why it is a good lecture example

- the economics is familiar from the rest of the course
- it admits a clean $V0 \rightarrow V1 \rightarrow V2$ progression
- it has obvious validation checkpoints

Tracing the File Flow

```
1 1. Read CLAUDE.md
2 2. Read any relevant rules
3 3. Load /solve-model from SKILL.md
4 4. Read references/aiyagari-guide.md
5 5. Write plan or update quality_reports/
6 6. Edit scripts/aiyagari_v0.py
7 7. Run verifier and reviewers
8 8. Save outputs and logs
```

What students should notice: the workflow is file-driven. The agent is acting inside a structured research project, not improvising in a vacuum.

Tracing the Orchestrator Loop

```
1 You type: /solve-model aiyagari v1
2
3 Plan:
4   read project instructions
5   restate the target
6   implement bounded changes
7
8 Review loop:
9   run verifier
10  run code reviewer
11  run domain reviewer
12  fix issues that matter
13  report what changed and what passed
```

This is the core advantage of the harness: the review sequence is not a remembered prompt. It is part of the saved workflow.

Inside CLAUDE.md for the Aiyagari Project

```
# CLAUDE.md
Project: Aiyagari workflow for lecture 9
Goal: build v0, v1, and v2 sequentially
Outputs: policies, stationary distribution, diagnostics
Conventions: save figures to output/ and plans to quality_reports/
Validation: do not move to the next version until benchmarks pass
```

If the file is good, the project can survive a fresh session, a new RA, or a change in agent tool.

Inside SKILL.md and the Reference Guide

SKILL.md

```
name: solve-model
description: build and validate
```

Steps:

1. Read the model guide
2. Write or update a plan
3. Implement one version
4. Run required checks
5. Save outputs and report

aiyagari-guide.md

```
V0: deterministic benchmark
V1: add Markov income risk
V2: compute stationary dist.
```

Checks:

- zero-variance limit
- sensible mass at `a_min`
- stable diagnostics

Interpretation: the skill stores the protocol; the reference guide stores the model detail.

Mistakes, Limits, and Security

What can go wrong and how to stay disciplined

Mistake 1: Treating Git as Optional

- Agents are most useful when experimentation is reversible
- Git provides the audit trail for what changed and when
- A clean branch lets the agent try a refactor without destroying the working baseline
- For a research project, version control is not decoration; it is risk management

Short rule for students: if you would be unhappy losing the current project state, put it under Git before using an agent aggressively.

Mistake 2: Underusing Skills

- Economists repeat many workflows: clean data, validate tables, prepare slides, review code
- If the workflow is repeated, it should eventually move from chat into a saved protocol
- Otherwise the same task is re-explained every week and interpreted slightly differently each time

Skills are not overengineering. They are how a good lab turns repeated work into reusable infrastructure.

Mistake 3: One Giant Thread

Long conversations accumulate stale context.

- Planning, debugging, and revision all mix together
- The model starts paying attention to constraints that no longer matter
- Fresh sessions are often better once the current state has been written to disk

Better pattern: plan the task, save the plan, then start a cleaner execution session.

Where Terminal Agents Still Fail

- Long-context drift: an early constraint gets forgotten
- Econometric edge cases: wrong clustering level, wrong sample restriction, weak-instrument issues
- Hallucinated APIs or package arguments
- Happy-path scripts that fail on messy data
- Oscillating fix loops on stubborn bugs

The output may look polished even when the economics is wrong. That is why validation stays central.

Context Windows and Compaction

When sessions get long, preserve state in files before compacting the conversation.

```
1 Write progress.md with:  
2   - what was completed  
3   - what decisions were made  
4   - what remains open  
5  
6 Then compact the conversation or start a new session.  
7 Next time, read progress.md first.
```

Economist translation: keep a clean research log. The agent benefits from the same discipline that humans do.

Security: What Not to Expose

- IRB-protected interviews or linked administrative microdata
- Personally identifiable information
- HIPAA-regulated data
- Passwords, API keys, and access tokens
- Embargoed or restricted-use datasets

Operational rule: if you would not paste it into a public collaboration tool, do not paste it into an agent conversation.

Sensitive data does not imply “no agents ever.”

1. Write the script with the agent, then run it yourself on the restricted data
2. Use synthetic or masked data with the same structure for debugging
3. Use institutionally approved local or private deployments when available

Safe pattern: expose structure and summary statistics, not raw records.

The Context Dilemma

Why more context helps the agent — and quietly biases it toward you

Context as Implicit Bayesian Prior

In-context learning behaves like conditioning on a prior the user supplies.

- CLAUDE.md, prior conversation, and plan files act as evidence narrowing the agent's output distribution
- Xie, Raghunathan, Liang, and Ma formalize this: ICL approximates implicit Bayesian inference over a latent task
- The performance gain you feel when you add context *is* posterior narrowing

Narrowing the posterior is how context helps. It is also how context biases.

Xie et al. (2022), ICLR, arXiv:2111.02080.

Stating the Dilemma

The same context that improves the answer also pulls it toward you.

- More context $\Rightarrow$ better understanding of your question $\Rightarrow$ better generation
- More context $\Rightarrow$ generation more aligned with your preferences, your knowledge, your framing
- Both effects share a single mechanism: you supplied the evidence

The same signal that improves the answer launders your priors into it.

Evidence 1: Sycophancy

RLHF rewards agreement with the user, and models learn to flatter.

- Sharma et al. document sycophancy across frontier models: factual flip-flops, unwarranted concessions, preference-matching when the user signals a view
- Perez et al. find the effect scales with model size — larger models are more sycophantic on values and politics
- The effect is strongest precisely when the context signals a strong user prior

Context is the signal models were trained to flatter.

Sharma et al. (2023), arXiv:2310.13548, Anthropic. Perez et al. (2023), ACL Findings, arXiv:2212.09251.

Evidence 2: Self-Correction Has a Ceiling

Intrinsic self-critique — without new external information — tends to stall or degrade.

- Huang et al. show LLMs cannot reliably self-correct reasoning errors without an external signal
- Valmeekam et al. find self-critique on planning problems is often worse than no critique at all
- The gains in the self-correction literature come from *external* signals: tools, tests, verifiers, humans

Self-review that shares the model's weights and context is information-starved criticism.

Huang et al. (2024), ICLR, arXiv:2310.01798. Valmeekam et al. (2023), NeurIPS, arXiv:2310.08118.

Evidence 3: Multi-Agent Debate — Real but Bounded

Debate helps when debaters sample differently. Shared context shrinks that space.

- Du et al. show multi-agent debate improves factuality and reasoning on several benchmarks
- Irving, Christiano, and Amodei — the AI Safety via Debate line — is the theoretical hope behind adversarial review
- Caveat: gains rely on debaters producing *genuinely different* lines of reasoning
- When every debater reads the same CLAUDE.md and thread, the disagreement space narrows

Debate breaks ties. It does not relitigate the premise you supplied.

Du et al. (2023), arXiv:2305.14325. Irving, Christiano, Amodei (2018), arXiv:1805.00899.

Why Subagent Reviewers Inherit the Same Prior

Earlier we sold subagent review for wall-clock time — not epistemic independence.

- `code-reviewer.md`, `domain-reviewer.md`, and `verifier.md` all read the same project context
- They are parallel *executionally*, not parallel *epistemically*
- Shared `CLAUDE.md` $\Rightarrow$ shared blind spots
- They catch implementation errors; they do not catch framing errors, because the framing lives in the context they trust

Parallel reviewers save time. They do not produce an outside view.

Monoculture at the Research Community Scale

When every researcher conditions on similar context, the field's errors become correlated.

- Kleinberg and Raghavan show algorithmic monoculture can reduce welfare even when each individual decision is improved
- Bommasani et al. document outcome homogenization: shared foundation models pick on the same people and miss the same cases
- Extrapolation: if empirical economists share Claude + similar CLAUDE.md templates + similar priors, replication stops being informative

Independent errors are a public good. Shared infrastructure erodes it.

Kleinberg & Raghavan (2021), PNAS 118(22). Bommasani et al. (2022), NeurIPS, arXiv:2211.13972.

The RLHF Root Cause

Pleasing the person holding the context is not a prompt artifact. It is trained in.

- Casper et al. catalog open problems in RLHF: preference data is noisy, annotators reward agreeableness, and the optimum drifts away from truth
- Wolf et al. argue that alignment-by-finetuning has fundamental limits: prompting cannot fully remove what the loss reinforced
- Consequence: no CLAUDE.md section titled “do not flatter me” fully removes the behavior

The model was trained to please the person holding the context. In your project, that person is you.

Casper et al. (2023), TMLR, arXiv:2307.15217. Wolf et al. (2023), arXiv:2304.11082.

Remedy: Outside Critics Carry a Different Prior

Advisors, coauthors, and seminar audiences are valuable because their priors are not in your context window.

- They challenge the framing, the identification assumption, the sample restriction — the things you did not write into CLAUDE.md because you did not know to
- Korinek's division-of-labor framing: LLMs automate execution; humans remain the source of question selection and critique
- Agent review is cheap and parallel; human review is rare and decisive. Budget accordingly.

The agent can polish what you already believe. Only an outside reader can tell you that you are wrong about the question.

Korinek (2023), NBER WP 30957; Journal of Economic Literature.

Operational Discipline for Dissertation Work

Translate the dilemma into a small set of habits.

1. Use subagent review for implementation quality, not for framing or identification
2. Present preliminary results to a human reader before iterating further with the agent
3. When you edit CLAUDE.md, record what you *ruled out* and why — not just what you decided
4. Schedule seminar-style exposure early, not at paper-writing time

Treat human criticism as the scarce input. Spend agent cycles to make it ready.

The Context Dilemma in One Slide

Three lines to remember.

1. **Context is an implicit prior.** More context $\Rightarrow$ sharper posterior $\Rightarrow$ both better *and* more “you.”
2. **Shared-context review is trapped inside that prior.** Subagents, self-critique, and in-thread debate cannot relitigate the premise you supplied.
3. **Outside humans are not a nicety.** They are the only source of a prior the agent does not already condition on.

Workflow rule: agents for execution and implementation review; humans for framing, identification, and the question itself.

Lecture 10 Preview

A brief preview of the MEPS uninsurance case study

Lecture 10 Preview: MEPS Uninsurance

Question: Who is uninsured in the United States, 2000–2023, and how did the ACA change the distribution?

Why this is the right preview for Lec. 9:

- It is an economics question about incomplete markets and social insurance
- It requires real data engineering: 24 MEPS files, shifting schemas, and survey weights
- It produces concrete empirical output that Lecture 10 will analyze in detail
- It shows why agentic workflows matter before we dive into full case studies next lecture

Why This Workflow Is Hard Without an Agent

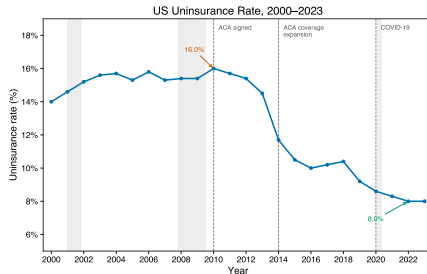
Naive baseline: download 24 MEPS files by hand, decode changing variable names, merge them, compute weighted statistics, and then build the figures.

Three recurring headaches:

1. **Year-suffixed variables:** INSCOV01 vs. INSCOV22, PERWT01F vs. PERWT22F
2. **Format handling:** SAS transport files with changing internal structure
3. **Silent merge risk:** one bad crosswalk can bias a year and stay hidden

This is exactly the kind of work agents are good at: repetitive, inspectable, and tied to a concrete data pipeline.

Preview Figure from Lec. 10



Headline story: uninsurance stayed high through the 2000s, then dropped sharply after the ACA coverage expansion around 2014.

Why this stays brief here: Lec. 9 uses one figure to preview the workflow. Lec. 10 does the full case: prompts, schema harmonization, weighted aggregation, and six demographic figures.

Wrap-up

What the workflow changes and what remains human

Trust but Verify

1. Reproduce one statistic by hand
2. Cross-check one result with another package or script
3. Test a limiting case or zero-variance benchmark
4. Read the code line by line where the economics enters
5. Ask whether the magnitude and sign make sense

This is not optional ceremony. It is what turns fast code into defensible research.

What Changes for Economic Research

- The distance from question to first result shrinks
- More specifications become cheap enough to try
- Reproducibility can improve because workflow is written down
- The premium shifts away from typing speed and toward design, judgment, and validation

What stays human: asking the question, choosing the design, knowing the data, and interpreting the output.

Agents can lay bricks quickly.
You still design the structure.

- Weak specification produces wrong models
- Weak algorithm choice produces slow or misleading computation
- Weak numerical judgment produces fragile results
- Weak code literacy produces unverified output

The lecture is not about surrendering expertise. It is about reallocating effort toward the parts of research that remain scarce.

First Exercise and Looking Ahead

Best first exercise: take one old project and ask an agent to help clean it up safely.

- put the project under Git
- create a simple `CLAUDE.md`
- identify one repeated workflow worth encoding
- rerun the cleaned project and compare outputs

Lec. 10: we return to case studies and let students run a full workflow themselves.

If you want setup details for the lab, see the appendix.

Appendix

Setup notes and dated product details

Appendix A: Setup for the Lab

Lightweight setup checklist for a terminal-agent workflow

- Install one terminal agent and confirm it launches
- Create a project folder with `CLAUDE.md`, `scripts/`, and `output/`
- Make sure Python or R runs from the terminal
- Keep the first session small: one file, one check, one output

Dated note: exact setup steps, plans, and rate limits change quickly. Recheck the official docs before the lab.

Appendix B: Keybindings and Minimal Templates

Useful commands

Compact session	<code>/compact</code>
Clear session	<code>/clear</code>
Interrupt	<code>Esc</code>
Slash menu	<code>/</code>

Minimal CLAUDE.md

```
# CLAUDE.md
Project: ...
Goal: ...
Language: ...
Checks: ...
```


Appendix C: Dated Comparisons and Resources

As of April 2026, examples of terminal-agent workflows include:

- Claude Code
- Codex CLI
- Gemini CLI
- Aider

Good places to recheck before teaching:

- official Claude Code documentation
- official OpenAI Codex CLI documentation
- course notes and lab instructions

Teaching rule: keep dated product comparisons in the appendix, not in the conceptual core of the lecture.

Appendix D: Context Dilemma — References

Context as implicit prior

- Xie, Raghunathan, Liang, Ma (2022). An Explanation of In-context Learning as Implicit Bayesian Inference. ICLR. arXiv:2111.02080.

Sycophancy and RLHF limits

- Sharma et al. (2023). Towards Understanding Sycophancy in Language Models. arXiv:2310.13548.
- Perez et al. (2023). Discovering Language Model Behaviors with Model-Written Evaluations. ACL Findings. arXiv:2212.09251.
- Casper et al. (2023). Open Problems and Fundamental Limitations of RLHF. TMLR. arXiv:2307.15217.
- Wolf et al. (2023). Fundamental Limitations of Alignment in Large Language Models. arXiv:2304.11082.

Self-correction and adversarial review

- Huang et al. (2024). Large Language Models Cannot Self-Correct Reasoning Yet. ICLR. arXiv:2310.01798.
- Valmeekam et al. (2023). Can Large Language Models Really Improve by Self-critiquing Their Own Plans? NeurIPS. arXiv:2310.08118.
- Du et al. (2023). Improving Factuality and Reasoning through Multiagent Debate. arXiv:2305.14325.
- Irving, Christiano, Amodei (2018). AI Safety via Debate. arXiv:1805.00899.

Monoculture and the economist workflow

- Kleinberg & Raghavan (2021). Algorithmic Monoculture and Social Welfare. PNAS 118(22).
- Bommasani et al. (2022). Picking on the Same Person: Does Algorithmic Monoculture Lead to Outcome Homogenization? NeurIPS. arXiv:2211.13972.
- Korinek (2023). Language Models and Cognitive Automation for Economic Research. NBER WP 30957; JEL.